

```
<pre>
BIP: 350
Layer: Applications
Title: Bech32m format for v1+ witness addresses
Authors: Pieter Wuille <pieter@wuille.net>
Status: Deployed
Type: Specification
Assigned: 2020-12-16
License: BSD-2-Clause
Discussion: 2021-01-05: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-January/018338.html [bitcoin-dev] Bech32m BIP: new checksum, and usage for segwit address
Replaces: 173
</pre>
```

==Introduction==

===Abstract===

This document defines an improved variant of Bech32 called `Bech32m`, and amends BIP173 to use Bech32m for native segregated witness outputs of version 1 and later. Bech32 remains in use for segregated witness outputs of version 0.

===Copyright===

This BIP is licensed under the 2-clause BSD license.

===Motivation===

[[bip-0173.mediawiki|BIP173]] defined a generic checksummed base 32 encoded format called Bech32. It is in use for segregated witness outputs of version 0 (P2WPKH and P2WSH, see [[bip-0141.mediawiki|BIP141]]), and other applications.

Bech32 has an unexpected [https://github.com/sipa/bech32/issues/51 weakness]: whenever the final character is a 'p', inserting or deleting any number of 'q' characters immediately preceding it does not invalidate the checksum. This does not affect existing uses of witness version 0 BIP173 addresses due to their restriction to two specific lengths, but may affect future uses and/or other applications using the Bech32 encoding.

This document addresses that by specifying Bech32m, a variant of Bech32 that mitigates this insertion weakness and related issues.

==Specification==

We first specify the new checksum algorithm, and then document how it should be used for future Bitcoin addresses.

===Bech32m===

Bech32m modifies the checksum of the Bech32 specification, replacing the constant `1` that is xored into the checksum at the end with `0x2bc830a3`. The resulting checksum verification and creation algorithm (in Python, cf. the code in [https://github.com/bitcoin/bips/blob/master/bip-0173.mediawiki#Bech32|BIP173 Bech32 section]):

```
<pre>
BECH32M_CONST = 0x2bc830a3

def bech32m_polymod(values):
    GEN = [0x3b6a57b2, 0x26508e6d, 0x1ea119fa, 0x3d4233dd, 0x2a1462b3]
    chk = 1
```

```

for v in values:
    b = (chk >> 25)
    chk = (chk & 0xffffffff) << 5 ^ v
    for i in range(5):
        chk ^= GEN[i] if ((b >> i) & 1) else 0
return chk

def bech32m_hrp_expand(s):
    return [ord(x) >> 5 for x in s] + [0] + [ord(x) & 31 for x in s]

def bech32m_verify_checksum(hrp, data):
    return bech32m_polymod(bech32m_hrp_expand(hrp) + data) == BECH32M_CONST

def bech32m_create_checksum(hrp, data):
    values = bech32m_hrp_expand(hrp) + data
    polymod = bech32m_polymod(values + [0,0,0,0,0,0]) ^ BECH32M_CONST
    return [(polymod >> 5 * (5 - i)) & 31 for i in range(6)]

```

All other aspects of Bech32 remain unchanged, including its human-readable parts (HRPs).

A combined function to decode both Bech32 and Bech32m simultaneously could be written using:

```


```

class Encoding(Enum):
 BECH32 = 1
 BECH32M = 2

def bech32_bech32m_verify_checksum(hrp, data):
 check = bech32_polymod(bech32_hrp_expand(hrp) + data)
 if check == 1:
 return Encoding.BECH32
 elif check == BECH32M_CONST:
 return Encoding.BECH32M
 else:
 return None

```


```

which returns either None for failure, or one of the BECH32 / BECH32M enumeration values to indicate successful decoding according to the respective standard.

===Addresses for segregated witness outputs===

Version 0 outputs (specifically, P2WPKH and P2WSH addresses) continue to use Bech32^{[\[1\]](#)} Why not permit both Bech32 and Bech32m for v0 addresses? Permittin g both encodings reduces the error detection capabilities (it makes it equivalent to only have 29 bits of checksum).^{[\[2\]](#)} as specified in BIP173. Addresses for segregated witness outputs version 1 through 16 use Bech32m. Again, all other aspects of the encoding remain the same, including the 'bc' HRP.

To generate an address for a segregated witness output:

- * If its witness version is 0, encode it using Bech32.
- * If its witness version is 1 or higher, encode it using Bech32m.

To decode an address, client software should either decode with both a Bech32 and a Bech32m decoder^{[\[3\]](#)} Can a single string simultaneously be valid as Bech32 and Bech32m? No, a valid Bech32 and Bech32m string will always differ by at least 3 characters if they are the same length.^{[\[4\]](#)}, or use a decoder that supports both simultaneously. In both cases, the address decoder has to verify that

the encoding matches what is expected for the decoded witness version (Bech32 for version 0, Bech32m for others).

The following code demonstrates the checks that need to be performed. Refer to the Python code linked in the reference implementation section below for full details of the called functions.

```
<pre>
def decode(hrp, addr):
    hrpgot, data, spec = bech32_decode(addr)
    if hrpgot != hrp:
        return (None, None)
    decoded = convertbits(data[1:], 5, 8, False)
    # Witness programs are between 2 and 40 bytes in length.
    if decoded is None or len(decoded) < 2 or len(decoded) > 40:
        return (None, None)
    # Witness versions are in range 0..16.
    if data[0] > 16:
        return (None, None)
    # Witness v0 programs must be exactly length 20 or 32.
    if data[0] == 0 and len(decoded) != 20 and len(decoded) != 32:
        return (None, None)
    # Witness v0 uses Bech32; v1 through v16 use Bech32m.
    if data[0] == 0 and spec != Encoding.BECH32 or data[0] != 0 and spec != Encoding.BECH32M:
        return (None, None)
    # Success.
    return (data[0], decoded)
</pre>
```

'''Error locating'''

Bech32m, like Bech32, does support locating^{<ref>'''What about error correction?'''} As explained in BIP173, introducing error correction reduces the ability to detect errors. While it is technically possible to correct a small number of errors due to Bech32(m)'s nature as a BCH code, implementations should refrain from using this for more than indicating where an error may be present.^{</ref>} the positions of a few substitution errors. To combine this functionality with the segregated witness addresses proposed by this document, simply try locating errors for both Bech32 and Bech32m. If only one finds error locations, report that one. If both do (which should be very rare), there are a number of options:

- * Report the one that needs fewer corrections (if they differ).
- * Eliminate the response(s) that are inconsistent. Any symbol that isn't on an error location can be checked. For example, if the witness version symbol is not an error location, and it doesn't correspond to the specification used (0 for Bech32, 1+ for Bech32m), that response can be eliminated.

See the fancy Javascript decoder below for example of the above.

==Compatibility==

This document introduces a new encoding for v1 segregated witness outputs and higher versions. There should not be any compatibility issues on the receiver side; no wallets are creating v1 segregated witness addresses yet, as the output type is not usable on mainnet.

On the other hand, the Bech32m proposal breaks forward-compatibility for sending to v1 and higher version segregated witness addresses. This incompatibility is [https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2020-October/018236.html intentional]. An alternative design was [https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2019-November/017460.html considered] where Bech32 remained i

No string can be simultaneously valid Bech32 and Bech32m, so the above examples also serve as invalid test vectors for Bech32.

The following string are not valid Bech32m (with reason for invalidity):

- * 0x20 + <tt>1xj0phk</tt>: HRP character out of range
- * 0x7F + <tt>1g6xzxy</tt>: HRP character out of range
- * 0x80 + <tt>1vctc34</tt>: HRP character out of range
- * <tt>an84characterslonghumanreadablepartthatcontainstheexcludedcharactersbioandnumber11d6pts4</tt>: overall max length exceeded
- * <tt>qyrz8wqd2c9m</tt>: No separator character
- * <tt>1qyrz8wqd2c9m</tt>: Empty HRP
- * <tt>y1b0jsk6g</tt>: Invalid data character
- * <tt>lt1igcx5c0</tt>: Invalid data character
- * <tt>in1muywd</tt>: Too short checksum
- * <tt>mm1crxm3i</tt>: Invalid character in checksum
- * <tt>auls5cgom</tt>: Invalid character in checksum
- * <tt>M1VUXWEZ</tt>: checksum calculated with uppercase form of HRP
- * <tt>16plkw9</tt>: empty HRP
- * <tt>1p2gdwpf</tt>: empty HRP

===Test vectors for v0-v16 native segregated witness addresses===

The following list gives valid segwit addresses and the scriptPubKey that they translate to in hex.

- * <tt>BC1QW508D6QEJXTDG4Y5R3ZARVARY0C5XW7KV8F3T4</tt>: <tt>0014751e76e8199196d454941c45d1b3a323f1433bd6</tt>
- * <tt>tb1qrp33g0q5c5txsp9arysr4k6zdkfs4nce4xj0gdcccfvpsxf3q0sl5k7</tt>: <tt>00201863143c14c5166804bd19203356da136c985678cd4d27a1b8c6329604903262</tt>
- * <tt>bc1pw508d6qejxtgd4y5r3zarvary0c5xw7kw508d6qejxtgd4y5r3zarvary0c5xw7kt5nd6y</tt>: <tt>5128751e76e8199196d454941c45d1b3a323f1433bd6751e76e8199196d454941c45d1b3a323f1433bd6</tt>
- * <tt>BC1SW50QGDZ25J</tt>: <tt>6002751e</tt>
- * <tt>bc1zw508d6qejxtgd4y5r3zarvaryvaxxpcs</tt>: <tt>5210751e76e8199196d454941c45d1b3a323</tt>
- * <tt>tb1qqqqqp399et2xygdj5xreqhjvcmzhxw4aywxecjdzew6hylvgsesrxh6hy</tt>: <tt>0020000000c4a5cad46221b2a187905e5266362b99d5e91c6ce24d165dab93e86433</tt>
- * <tt>tb1pqqqqqp399et2xygdj5xreqhjvcmzhxw4aywxecjdzew6hylvgsesf3hn0c</tt>: <tt>5120000000c4a5cad46221b2a187905e5266362b99d5e91c6ce24d165dab93e86433</tt>
- * <tt>bc1p0xlxlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vqzk5jj0</tt>: <tt>512079be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798</tt>

The following list gives invalid segwit addresses and the reason for their invalidity.

- * <tt>tc1p0xlxlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vq5zuyut</tt>: Invalid id human-readable part
- * <tt>bc1p0xlxlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vqh2y7hd</tt>: Invalid id checksum (Bech32 instead of Bech32m)
- * <tt>tb1z0xlxlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vqgl7rf</tt>: Invalid id checksum (Bech32 instead of Bech32m)
- * <tt>BC1S0XLXLHEMJA6C4DQV22UAPCTQUPFHLXM9H8Z3K2E72Q4K9HCZ7VQ54WELL</tt>: Invalid id checksum (Bech32 instead of Bech32m)
- * <tt>bc1qw508d6qejxtgd4y5r3zarvary0c5xw7kemeawh</tt>: Invalid checksum (Bech32m instead of Bech32)
- * <tt>tb1q0xlxlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vq24jc47</tt>: Invalid id checksum (Bech32m instead of Bech32)
- * <tt>bc1p38j9r5y49hruaue7wxjce0updquyuyx0kh56v8s25huc6995vvpql3jow4</tt>: Invalid id character in checksum
- * <tt>BC130XLXLHEMJA6C4DQV22UAPCTQUPFHLXM9H8Z3K2E72Q4K9HCZ7VQ7ZWS8R</tt>: Invalid id witness version
- * <tt>bc1pw5dgrnzv</tt>: Invalid program length (1 byte)
- * <tt>bc1p0xlxlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7v8n0nx0muaewav253zgeav</tt>: Invalid program length (41 bytes)

- * `<tt>BC1QR508D6QEJXTDG4Y5R3ZARVARYV98GJ9P</tt>`: Invalid program length for witness version 0 (per BIP141)
- * `<tt>tb1p0xlxvlhemja6c4dq22uapctqpfhlxm9h8z3k2e72q4k9hcz7vq47Zagq</tt>`: Mixed case
- * `<tt>bc1p0xlxvlhemja6c4dq22uapctqpfhlxm9h8z3k2e72q4k9hcz7v07qwwzcrf</tt>`: zero padding of more than 4 bits
- * `<tt>tb1p0xlxvlhemja6c4dq22uapctqpfhlxm9h8z3k2e72q4k9hcz7vpggkg4j</tt>`: Non-zero padding in 8-to-5 conversion
- * `<tt>bc1gm9yu</tt>`: Empty data section

==Appendix: checksum design & properties==

Checksums are used to detect errors introduced into data during transfer. A hash function-based checksum such as Base58Check detects any type of error uniformly, but not all classes of errors are equally likely to occur in practice. Bech32 prioritizes detection of substitution errors, but improving detection of one error class inevitably worsens detection of other error classes. During the design of Bech32, it was assumed that other simple error patterns beside substitutions would have a similar detection rate as in a hash function-based design, and detection would only be worse for complex, impractical errors. The discovered insertion weakness shows that this is not the case.

For Bech32m, we aim to retain Bech32's guarantees for substitution errors, but make sure that other common errors don't perform worse than a hash function-based checksum would. To make sure the new standard is easy to implement, we restrict the design space to only amending the final constant that is xored in, as it was [observed](https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2019-December/017521.html) that is sufficient to mitigate the 'q' insertion issue while retaining the intended substitution error detection. In what follows, we explain how the new constant `'0x2bc830a3'` was chosen.

===Error patterns & detection probability===

We define an error pattern as a sequence of first one or more deletions, then swaps of adjacent characters, followed by substitutions, insertions, and duplications, in that order, all in specific positions, applied to a string with valid checksum that is otherwise randomly chosen. For insertions and substitutions we assume a uniformly random new character. For example, "delete the 17th character, swap the 11th character with the 12th character, and insert a random character in the 24th position" is an error pattern. "Replace the 43rd through 48th character with 'aardvark'" is not a valid error pattern, because the new characters are not random and there is no reason why this particular string is more likely than any other to be substituted.

A hash function-based checksum design with a 30-bit hash would have a probability of incorrectly accepting equal to 2^{-30} , for every error pattern. Bech32 has a probability of 0 to incorrectly accept error patterns consisting of up to 4 substitutions—they are always detected. The 'q'-insertion issue shows that for Bech32 a simple error pattern ("insert a random character in the penultimate position") with probability 2^{-10} exists: it requires the final character to be 'p' (leaving only 1 in 32 strings), and requires the inserted character to be 'q' (permitting only 1 of 32 possible inserted characters).

Note that the choice of 'what' the error pattern is (which types of errors, and where) isn't part of our probabilities: we try to make sure that 'every' pattern behaves well, not just randomly chosen ones, because presumably humans make some kinds of errors more than others, and we cannot easily model which ones.

===Detection properties of Bech32m===

The table below shows the error detection properties of Bech32m, and a comparison with Bech32. The code used for this analysis can be found [https://gist.github.com/sipa/14c248c288c3880a3b191f978a34508e#file-const_analysis-cpp here]. Every row specifies one error pattern via the constraints in the left four columns. The remaining columns report what percentage of those patterns have certain probabilities of not being detected. The columns are:

- * `''errors''` The maximum number of individual errors considered
- * `''of type''` What type of errors are considered (either "subst. only" for just substitutions, or "any" to also include deletions, swaps, insertions, and duplications)
- * `''window''` The maximum size of the window in which the errors have to occur<ref>''What is an error pattern's window size?'' The window size of an error pattern is the length of the smallest consecutive range of characters that contains all modified characters (on input or output; whichever is larger). For example, an error pattern that turns "abcdef" into "accdbef" has a window size of 4, as it is replacing "bcd" with "ccdb", a 4 character string. Window size is only meaningful when the pattern consists of two or more errors.</ref>
- * `''code/verifier''` Whether this line is about Bech32 or Bech32m encoded strings, and whether those are evaluated regarding their probability of being accepted by either a Bech32 or a Bech32m verifier.<ref>''Why do we care about probability of accepting Bech32m strings in Bech32 verifiers?'' For applications where Bech32m replaces an existing use of Bech32 (such as segregated witness addresses), we want to make sure that a Bech32m string created by new software won't be erroneously accepted by old software that assumes Bech32 - even when a small number of errors were introduced as well.</ref><ref>''Should we also take into account failures that occur due to taking a valid Bech32m string, and after errors it becoming acceptable to a Bech32 verifier?'' This situation may in theory occur for segregated witness addresses when errors occur that change the version number in a v1+ address to v0. Due to the specificity of this type of error, plus the additional constraints that apply for v0 addresses, this is both unlikely and hard to analyze.</ref>
- * `''error patterns with failure probability''` For each probability (`''0''`, `''2<sup>-30</sup>''`, `''2<sup>-25</sup>''`, `''2<sup>-20</sup>''`, `''2<sup>-15</sup>''`, and `''2<sup>-10</sup>''`) this reports what percentage of error patterns restricted by the constraints in the previous columns have those probabilities of being incorrectly accepted.

The properties are divided into two classes: those that hold over all strings when averaged over all possible HRPs (human readable parts), and those specific to the "bc1" HRP with the length restrictions imposed by segregated witness addresses<ref>''What restrictions were taken into account for the "bc1"-specific analysis?'' The minimum length (due to witness programs being at least 2 bytes), the maximum length (due to witness programs being at most 40 bytes), and the fact that the witness programs are a multiple of 8 bits. The fact that the first data symbol cannot be over 16, or that the padding has to be 0, is not taken into account.</ref>.

```
{| class="wikitable"
| rowspan="2" | errors
| rowspan="2" | of type
| rowspan="2" | window
| rowspan="2" | code/verifier
| colspan="6" | error patterns with failure probability
|-
| ''0'' !! ''2<sup>-30</sup>'' !! ''2<sup>-25</sup>'' !! ''2<sup>-20</sup>'' !! ''2<sup>-15</sup>'' !! ''2<sup>-10</sup>''
|-
| colspan="10" | Properties averaged over all HRPs
|-
| &leq; 4 || only subst. || any || rowspan="6" | Bech32m/Bech32m || 100.00%
|| colspan="5" | none<sup>(a)</sup>
```

-	any	any	≤ 4	56.16%	43.84%	colspan="4" none^(b)				
-	≤ 2	any	≤ 68	7.71%	92.28%	colspan="4" none^(b)				
-	≤ 2	any	any	7.79%	92.20%	0.004%	colspan="3" none^(b)			
-	≤ 3	any	≤ 69	7.73%	92.23%	0.033%	^(d) colspan="3" none^(b)			
-	≤ 3	any	any	7.77%	92.19%	0.034%	0.000065%	colspan="2" none^(b)		
-	≤ 4	only subst.	any	rowspan="6" Bech32/Bech32				100.00%		
-	any	any	≤ 4	54.00%	43.84%	1.08%	0.90%	0.17%		
-	≤ 2	any	≤ 68	4.59%	92.29%	1.09%	1.01%	0.99%		
-	≤ 2	any	any	4.58%	92.21%	1.11%	1.04%	1.02%		
-	≤ 3	any	≤ 69	6.69%	92.23%	0.56%	0.48%	0.041%		
-	≤ 3	any	any	6.66%	92.19%	0.59%	0.52%	0.041%		
-	0	-	-	rowspan="3" Bech32m/Bech32				100.00%		
-	1	any	-	46.53%	53.46%	colspan="4" none^(b)				
-	≤ 2	any	any	22.18%	77.77%	0.048%	colspan="3" none^(b)			
-	colspan="10" Properties for segregated witness addresses with HRP "bc"									
-	≤ 4	only subst.	any	rowspan="6" Bech32m/Bech32m				100.00%		
-	1	any	-	24.34%	75.66%	colspan="4" none^(c)				
-	≤ 2	any	≤ 28	16.85%	83.15%	colspan="4" none^(c)				
-	any	any	≤ 4	74.74%	25.25%	0.0016%	colspan="3" none^(c)			
-	≤ 2	any	any	15.72%	84.23%	0.039%	0.0053%	colspan="2" none^(c)		
-	≤ 3	any	any	13.98%	85.94%	0.078%	0.00063%	colspan="2" none^(c)		

-									
≤ 4		only subst.		any		rowspan="6"> Bech32/Bech32		100.00%	
colspan="5"> none									
-									
1		any		-		14.63%		75.71%	
2.38%									
-									
≤ 2		any		≤ 28		14.22%		83.15%	
0.054%									
-									
any		any		≤ 4		73.23%		25.26%	
0.0064%									
-									
≤ 2		any		any		12.79%		84.24%	
0.041%									
-									
≤ 3		any		any		13.00%		85.94%	
0.00067%									
-									
≤ 3		only subst.		any		rowspan="6"> Bech32m/Bech32		100.00%	
colspan="5"> none ^(c)									
-									
1		any		-		70.89%		29.11%	
70.89%									
-									
≤ 2		any		any		36.12%		63.79%	
0.092%									
-									
≤ 2		any		any		0.00049%		c	
0.00049%									
}									

The numbers in this table, as well as a comparison with the numbers for the '1' constant and earlier proposed improved constants, can be found [https://gist.github.com/sipa/14c248c288c3880a3b191f978a34508e#file-results_final-txt here].

===Selection process===

The details of the selection process can be found [<https://gist.github.com/sipa/14c248c288c3880a3b191f978a34508e> here], but in short:

- * Start with the set of all '2³⁰-1' constants different from Bech32's '1'. All of these satisfy the properties marked ^(a) in the table above.
- * Through exhaustive analysis, reject all constants that do not exhibit the properties<ref>'How were the properties to select for chosen?'</ref> All these properties are as strong as they can be without rejecting every constant: rejecting constants with lower probabilities, or more errors, or wider windows all result in nothing left.</ref> marked ^(b) in the table above (e.g. all constants that permit any error pattern of 2 errors or less in a window of 68 characters or less with a detection probability '≥ 2⁻²⁰</sup>'). This selection leaves us with 12054 candidates.
- * Reject all constants that do not exhibit the ^(c) properties in the table above<ref>'Why optimize for segregated witness addresses (with HRP "bc1") specifically?'</ref> Our analysis for generic HRP has limitations (see the detailed description [https://gist.github.com/sipa/14c248c288c3880a3b191f978a34508e#file-bech32m_mail-txt here], under "Technical details"). We optimize for generic usage first, but optimize for segregated witness addresses as a tiebreaker.</ref>. This leaves us with 79 candidates.
- * Finally, select the candidate that minimizes the number of error classes matching ^(d) in the table above as a final tiebreaker. The result is the single constant '0x2bc830a3'.

===Footnotes===

<references />

==Acknowledgements==

Thanks to Greg Maxwell for doing most of the computation for code selection and analysis, and comments.
Thanks to Mark Erhardt for help with writing and editing this document.
Thanks to Rusty Russell and others on the bitcoin-dev list for the discussion around intentionally breaking compatibility with existing senders, which is used in this specification.